

Comparing the WMS real time algorithm with AIP predictive schedulers

D J Parrott and N Azarmi

This paper discusses the approach for comparing the performance of predictive manpower-scheduling with that of the Workforce Management System (WMS) real time algorithm (RTA). The former is based on advanced information processing (AIP) techniques and the latter on operations research (OR). Preliminary results of experimental comparisons between WMS and a predictive scheduling program using the AIP technique of simulated annealing are reported

1. Introduction

The motivation behind this research has been the knowledge that predictive schedulers provide the opportunity for global and local optimisation, and consequently the educated belief is that this will lead to better quality schedules. Now, however, it is becoming increasingly important to provide concrete evidence that it is not merely an educated belief, but that advantages can be gained using predictive schedulers.

The paper assumes a minimal familiarity with the WMS (Work Management System) manpower scheduling system [1] and the WEST scenario test harness developed within the Intelligent Systems Unit [2].

- The WMS RTA runs periodically during a working day, associating outstanding work with its pool of engineers. When an engineer requests work, a firm job-allocation is made using the last job associated with that engineer. Prior to a firm allocation, the WMS may scrap job/engineer associations in favour of new associations which, in the light of new information, are likely to yield better results.
- WEST is a simulation environment (the acronym stands for work and engineer simulation tool) designed to drive WMS in order to tune the latter's parameter settings. WEST has recently been adapted to drive other kinds of job scheduling programs. Therefore, WEST can be used as a basis for experimental comparisons of these with the WMS.

The remainder of this paper is organised as follows. Section 2 discusses the range of schedules which can be generated by these scheduling tools. The objective is to

understand how different classes of schedules are applied, in order to make fair comparisons between the schedulers which produce them. Section 3 contains a discussion of comparison metrics. In section 4 an outline description is given of the experimental system used to produce the results reported in this paper. Key issues pertaining to the experimental comparison of resource schedulers are discussed. The experimental comparison of the WMS RTA with a predictive scheduler based on simulated annealing is presented in section 5. Finally, conclusions are drawn in section 6.

2. Schedules for comparison

For sensible comparisons, it is clearly necessary to compare like with like. One of the problems faced when comparing the WMS RTA with predictive schedulers is that none of our predictive schedulers yet operate in a dynamic mode. They all take fixed information for the jobs and engineers to be scheduled over a whole day and schedule the day in advance. By contrast, the WMS RTA schedules job allocations for the current time only, repeating its scheduling task every five minutes. Attempts to resolve this issue are contained in the following sections.

2.1 Fully predictive schedules

Predictive schedulers work on the assumption that they have complete knowledge of the jobs and engineers present throughout the working day. The studies undertaken by the University of Strathclyde [3, 4] provide a large number of these schedules, based on randomly generated problems.

For the purposes of comparison, fully predictive schedules will be identified by the symbol S_p and the associated cost by the symbol $C(S_p)$ (see section 3 for further

detail on schedule cost). The measurement $C(S_p)$ is biased in favour of predictive schedulers because a complete knowledge of all events is assumed; a state which would very rarely, if ever, be achieved in practice.

2.2 Using predictive schedules in a dynamic environment

The predictive schedule S_p can be used to drive an experiment based on the WEST scenario test harness. Using WEST's abilities to randomly perturb expected events, the effectiveness of S_p can be examined. S_p contains a sequence of job allocations for each engineer. As the day progresses, jobs will be allocated from this sequence. If job appointment times are missed then jobs will either be done at the wrong time or can be deleted from the sequence, whichever is most appropriate.

Throughout the working day, a log is taken of jobs performed by the engineers. At the end of the day, the completed log can be treated as a schedule, S_a . The costfunction used to measure the effectiveness of predictive schedules is applied to S_a , giving its cost, $C(S_a)$. With minimal perturbations, the job sequence specified predictively will be suitable for the dynamically changing environment. However, it may be that certain jobs could not be done at the right time and, consequently, penalties incurred. Therefore, as perturbations increase the experiment is biased against predictive schedulers which have no capability to react to dynamic changes.

2.3 Making predictive schedules dynamic

To cope with dynamically changing environments, a predictive scheduler would have to be augmented with a dynamic rescheduling capability (see section 4.3 for a full explanation of this process). The scheduler would then be able to adjust its schedule if unexpected events occur. Again the allocations made can be logged and the completed log treated as the final schedule, S_d . The cost $C(S_d)$ of the dynamically constructed schedule is then measured directly.

2.4 Schedules generated by the WMS RTA

Job and engineer data is taken from predictive scheduling experiments and the WMS used to make allocations in real time. To derive a schedule suitable for comparisons with other schedulers, jobs are fed into the RTA at appropriate times and a log is taken of the real time allocations. As before, the completed log can be treated as a schedule, S_w . The cost-function used to measure the effectiveness of predictive schedules is applied to the WMS schedule to measure the cost, $C(S_w)$.

Ideally, $C(S_w)$ should be compared against $C(S_d)$. However, for the preliminary experiments presented here,

only predictive schedulers are used, thus limiting comparisons to those between $C(S_w)$, $C(S_p)$ and $C(S_a)$. To make the comparison as fair as possible, jobs can be restricted to appointments only (i.e. all jobs are known in advance by the WMS and have appointment timewindows throughout the day). Thus both types of scheduler have advance knowledge of all jobs. The experimental results will demonstrate whether or not improvements are obtained in schedule quality by utilising the global optimisations available to the predictive schedulers. Given that WMS was designed for only a small percentage of appointed jobs, further experiments may be necessary to ensure that the exclusive use of appointed jobs does not unfairly bias the experiment against WMS.

3. Metrics for comparison

To compare scheduling systems, schedules are produced by running different schedulers on the same input data and comparing the costs of the resultant schedules. This raises the questions:

- how is the cost of a schedule measured?
- what does it mean to have a cost for a schedule?

In this section a number of pitfalls are discussed and possible solutions are suggested.

3.1 Objective functions

Many scheduling techniques require an objective cost function to be defined, such as:

$$\text{Cost}(\text{Schedule}) = \text{Cost}(\text{overtime}) + \text{Cost}(\text{travel}) + \text{Cost}(\text{missed jobs}).$$

where the costs of overtime, travel, and missed jobs are determined according to factors such as wages paid for overtime, time spent travelling, and penalties associated with failing to perform various classes of job. The function reported in Brind et al [5] specialises the generic example to:

$$\text{Cost}(\text{Schedule}) = \text{Distance}(\text{travel}) + (600 \times \text{Duration}(\text{missed jobs})).$$

Objective cost functions are useful for rapidly estimating costs while the schedule is under construction. Key factors forming the parameters of the objective function can often be represented numerically prior to execution (e.g. in terms of job and engineer definitions).

While an objective function is useful for making scheduling decisions, there are disadvantages for using the same function to determine the quality of the final schedule. Firstly, the result is typically a coefficient without units and may thus be difficult to interpret. Too much information is

lost in normalising into a single number all of the parameters affecting the quality of the solution. Secondly, can the objective function be trusted? Without a thorough analysis of the problem, it is possible to build a function which is biased towards or against certain regions of the solution space. If the same badly designed function is thereafter used to compare the final schedules then:

- the comparison is somewhat tautologous,
- a scheduler may appear to be performing well merely because it is suited to the region of the search space in which it is directed by the cost function.

3.2 What makes a schedule good?

A 'good' schedule is one which fulfils the requirements of its users. To determine the extent to which those requirements are met, there may be a number of questions to answer (none of which may resemble the objective cost function used by the scheduler itself). For example, it is suggested in Manning and Gell [6] that two factors are important in determining the quality of a schedule:

- the utilisation of resources,
- the grade of service (GoS).

Utilisation is defined in Manning and Gell [6] as the percentage of resources that are busy. In the case of manpower scheduling, utilisation might be calculated as:

$$\frac{\text{total hours executing jobs} + \text{total travel time}}{\text{total hours worked by engineers}} \times 100\%$$

The above expression includes travelling time between jobs. This would not, therefore, be a good cost-metric if the goal is to minimise travelling time. However, it does retain vital information about the use of the workforce and can be used, for example, to influence strategic planning issues where the size of the workforce is under review. Fig 1 Basic functionality of the allocator.

Grade of service can be defined as:

$$\frac{\text{number of satisfied job requests}}{\text{total number of job requests}} \times 100\%$$

The expression could be made more complex to incorporate levels of service agreement. Alternatively, grades of service may be calculated separately for different service classes.

There may be other requirements for a good schedule. To compare final schedules properly, sufficient information should be logged such that all required measurements of quality can be made. Comparisons can then be based on any number of criteria. It is possible for one scheduler to perform better than another according to one set of criteria but worse according to a second set. It is left to the user to decide which is the more important.

4. A general-purpose job allocator

The functionality of a job-scheduling system can be broken down into two basic activities:

- the algorithm used to determine the schedule,
- the physical allocation of jobs to engineers.

This distinction is useful when different scheduling algorithms are to be compared. The functionality of the allocation activity is independent of the scheduling algorithm. A general-purpose allocator can therefore support the generic operations of all schedulers to be added to the basic system. In this section, the functionality of a generic allocator is outlined.

4.1 Basic allocator functionality

Figure 1 is a block diagram describing the required functionality of the allocator. The allocator maintains databases of job and engineer information and provides external interfaces to perform the following operations:

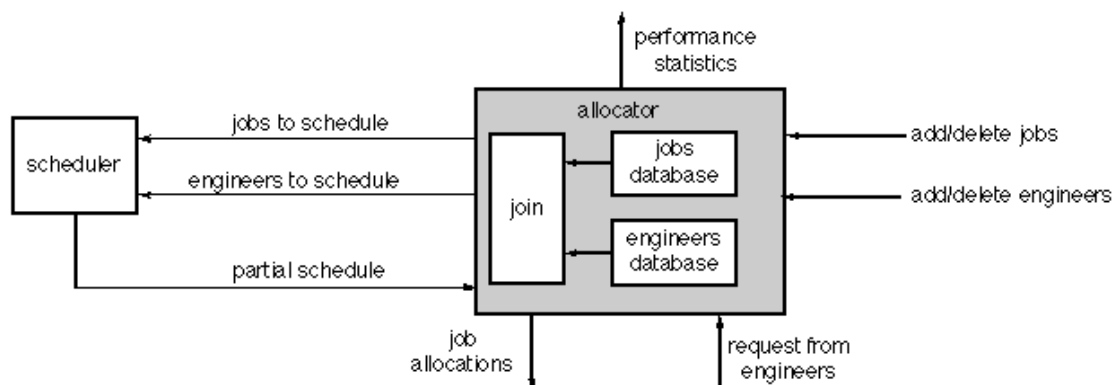


Fig 1 Basic functionality of the allocator.

- the addition and deletion of jobs and engineers,
- the supply of relevant job and engineer information to a scheduling algorithm which subsequently feeds the resultant (partial) schedule back to the allocator,
- the allocation of jobs to engineers upon receipt of job requests from engineers.

4.2 The allocator as a component of WEST

WEST interfaces with WMS in the manner shown in Fig 2. Jobs are entered into text files normally written by the customer service system (CSS) and communication between the WMS and the field access interface system (FAIS) is provided via remote procedure calls (RPC). Both methods allow WMS and WEST to execute on separate, networked machines. This is important for performance evaluation because WMS is able to run on a free processor and is thereby isolated from WEST's processor intensive operations.

In order to use unmodified, stand-alone scheduling programs with WEST, the allocator should communicate with schedulers via their existing mechanisms (typically files). Any attempt to use RPC for this communication link will require a good deal of modification to the I/O mechanism currently employed by the scheduler.

It was decided to run the allocator program independently of the WEST simulation in order to promote modularisation. Compare Figs 2 and 3 which illustrate the existing and extended architectures, respectively. Running the allocator as an independent program results in a spread

of responsibility across physical processors identical to that obtained with the WMS.

4.3 Predictive rescheduling in a dynamic environment

The experiments presented in this paper compare purely predictive schedulers with the WMS RTA. Currently, the predictive schedulers are being upgraded so that they operate more effectively in a dynamic environment [7]. One way to achieve this goal is via rescheduling. The predictive schedule is rebuilt at key times of the day in order to maintain up-to-date partial schedules. For each rebuild, new data is available which may be used to build a better partial schedule for the remaining portion of the day.

The rescheduling approach is similar to that taken by the WMS RTA. The difference is that the partial schedule calculated for each engineer extends only a small time into the future - the predictive scheduler is able to observe the whole of the remainder of the day and thus has the opportunity to make global optimisations. The WMS RTA runs once every five minutes, but the predictive schedulers may require longer time periods in order to generate good schedules. The period of rescheduling can be taken as an experimental factor. Strathclyde University's results are based on 10 minutes of CPU-time on a Sparc IPX to construct a one-day schedule [5].

Rescheduling must leave the initial portion of the existing schedule intact, up to the point when the rescheduling will be completed, as demonstrated in Fig 4

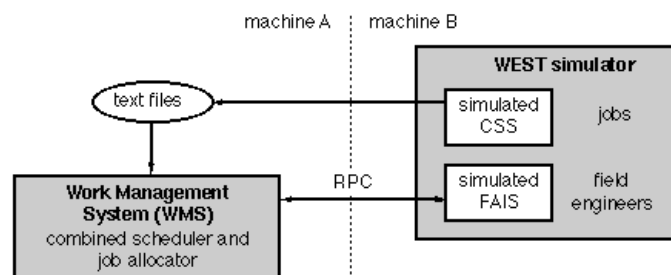


Fig 2 WMS as a component of WEST.

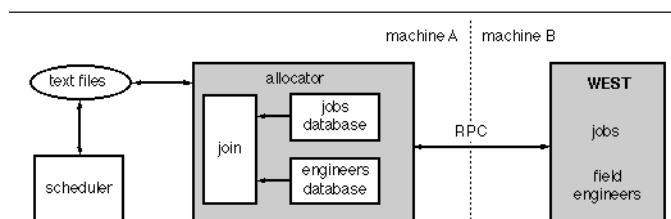


Fig 3 The extended architecture.

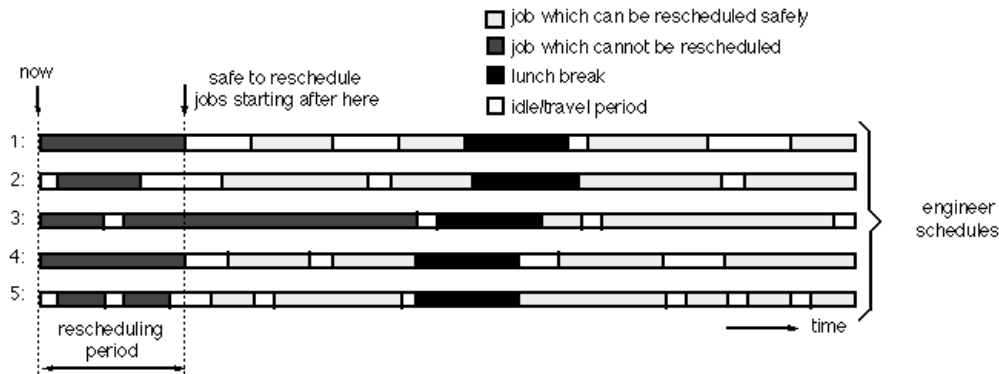


Fig 4 Ensuring the integrity of existing allocations while rescheduling.

The current schedule for engineer number 3 contains a job which is expected to start before the rescheduling period is complete and to extend until lunch-time. The job cannot be rescheduled because it is expected engineer 3 will request the job before the new schedule is built. Moreover, because engineer 3 will be executing the job until lunch time, the allocator tells the scheduler that the engineer is unavailable for work until after lunch.

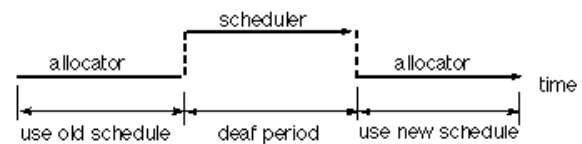


Fig 5 Scheduler and allocator running in line.

To implement the above rescheduling scheme, no modifications need to be made to the schedulers themselves. The initial portion of the schedule can be fixed by supplying:

- modified time windows for the engineers, i.e. their availability is artificially delayed until the end of the fixed part of the schedule,
- modified lists of jobs, i.e. removing those which occur in the fixed part of the schedule.

4.3.1 Synchronisation

Synchronisation is required to ensure that the current (re)scheduling activity is completed before schedule files are read and the next activity is begun. Synchronisation is achieved by running the scheduler in line with the allocator (see Fig 5).

Only a single instance of the scheduler may execute at any one time but the allocator will be deaf to new messages sent by WEST when the schedule is being rebuilt. To avoid the deaf period, the schedule can be rebuilt in the background, as a child process (see Fig 6).

Messages arriving from WEST while the scheduler is running are dealt with immediately. However, more complex synchronisation is required to avoid multiple instances of scheduling programs and to signal to the allocator when the new schedule file has been written.

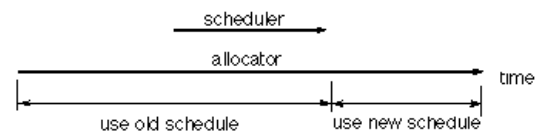


Fig 6 Scheduler as a child process.

4.4 Activating the scheduler

WEST has responsibility for time-tabling rescheduling activities. The allocator manages individual rescheduling operations, but the instruction to force rescheduling comes from WEST. Schedules are built in one or more of the following modes of operation.

4.4.1 Single shot operation — scheduling once only

For single-shot operation, all jobs must be known in advance. The jobs and available engineers are downloaded to the scheduling algorithm in one block and a schedule created. Any jobs entering the system after the scheduler has been started will not be assigned to engineers. Single-shot scheduling is intended for experimental purposes only.

4.4.2 Periodic and time-tabled rescheduling

The schedule is rebuilt either at regular time intervals throughout the day or at specific times. The latter is useful when the day is divided into unequal regions. For example,

the WMS system currently divides the day into four time-slots:

Slot	Start Time	End Time
1	08:00	10:30
2	10:30	13:00
3	13:00	15:30
4	15:30	17:00

4.4.3 Greedy rescheduling

The schedule is rebuilt when either of the following exceptions occur:

- reschedule every time an engineer requests a job,
- reschedule every time an engineer requests a job, but none was available.

4.5 Time-windows for jobs

Predictive schedulers produce schedules according to expected jobs durations. The allocator also uses expected durations for partial rescheduling but otherwise treats the schedule simply as an ordered sequence of jobs. However, each job possesses a time-window and, if jobs have overrun their expected time, subsequent jobs may fail by timing out. The allocator is responsible for time-out; timed out jobs cannot be allocated and are removed from the schedule.

Similarly, if a job has an earliest start-time, T_{early} , then it cannot commence before that time. The estimated travel time to the next job, T_{travel} , will be calculated and the allocation will not occur while $T_{now} + T_{travel} < T_{early}$ (where T_{now} is the current clock time).

The allocator is not responsible for changing the sequence of jobs in order to avoid failures due to timeout. This responsibility remains with the scheduling program.

4.6 Speeding up the clock

Time-driven simulation was chosen for the experiments reported in this paper because attempts are being made to compare dynamic scheduling programs operating within time-constraints. Using time-driven simulation, however, each experiment would take a whole day to complete if executed in real time. It is thus beneficial to speed up the experiments and, by careful planning, it is possible to obtain a constant factor speed-up without affecting the desired time-constraints.

In this section, problems in running experiments in faster than real time are identified. Firstly, the discussion focuses on the effects on experimental results of speeding up the overall experiment whilst retaining real time execution of the schedule program. Secondly, the discussion turns to potential problems with the scheduling tools themselves.

4.6.1 Biasing experimental results

The time taken to build a schedule depends on factors such as:

- the problem size (e.g. the number of engineers and jobs to schedule),
- fixed bounds on execution (e.g. for stochastic algorithms, a limit on the number of program cycles or the amount of CPU-time allowed).

The scheduling program cannot speed up in sympathy with the simulation. However, the real time interval between scheduling operations may be reduced to maintain a constant interval in experimental time, as demonstrated in Fig 7. As the real time interval becomes smaller, the scheduling operations begin to merge. In Fig 7, the limit is reached when the experiment is executed at 8 times real time.

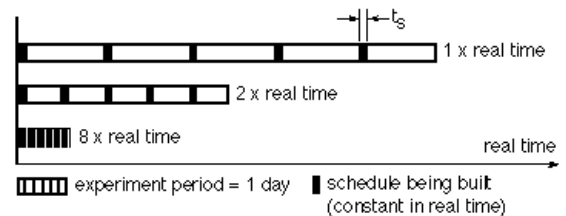


Fig 7 Speeding up experiments.

Consider an experiment running at x times real time where the real time taken by each scheduling operation is given by t_s . In relative, experiment time, the duration of a scheduling operation is given by $x.t_s$. Let the times of day at which scheduling is performed be $t_1, t_2, \dots, t_n$ (assuming n scheduling operations per day). Updates to the schedule will therefore be made at times $t_i + x.t_s$, for $1 \leq i \leq n$. Update times thus depend on the speed of the experiment; hence in relative, experiment time, updates are late when x is greater than 1.

It can be arranged that scheduling operations complete at fixed experiment times, $t_i + t_s$, by starting them earlier, at times $t_i - ((x-1).t_s)$. This also distorts the experiment, however, because the scheduler has less up-to-date information each time it runs; changes to the system occurring in the intervals $t_i - ((x-1).t_s)$ to t_i are carried over to subsequent scheduling operations.

Scheduling operations can be started at times between $t_i - ((x-1).t_s)$ and t_i to achieve varying degrees of late update and incomplete information. Whichever is chosen, it is required that:

- the experimental bias is not significant,

- the utility of running x times as many experiments in the available time is beneficial.

If either of these criteria fail to be satisfied, then experiments should be run in real time only.

5. Preliminary experiments

In this section comparisons between predictive schedulers and the WMS RTA are presented.

5.1 Experiment description

The experiment used for preliminary comparisons is described by the factors contained in Table 1.

Figure 8 shows the complete set of histograms describing the eight job profiles used in the experiments. Actual job durations are generated stochastically and are specified by a mean duration and standard deviation (a normal distribution is implemented at present). Estimated job durations are supplied programs; the estimates used in these experiments are shown in Fig 8 in conjunction with their associated means and standard deviations. (It should be noted that scheduling operations are performed over identical job-sets for all experiments by fixing the seed value of the pseudorandom number generator.)

The job profile histograms range over a 24 hour period running from midnight to midnight. The period is divided into 24 one-hour buckets and the normalised histograms describe the relative job-arrival rates. For each job type, the overall number of jobs for the whole period is displayed in parenthesis in Fig 8.

5.2 Results

The three workforce sizes presented to the schedulers represent under-resourcing, critical-resourcing, and overresourcing, respectively. Table 2 presents a summary of all schedules produced during the experiments.

Table 2 contains entries for four sets of schedules:

- schedules built predictively by the simulated annealer, using estimates of job durations,
- actual allocations obtained when the predictive SA schedules are used in a dynamic environment,
- actual allocations obtained when the WMS runs in a dynamic environment,
- schedules built predictively by the simulated annealer, using actual job durations.

Measurements of goodness are specified according to the following metrics:

- the number of engineers used to perform the work,
- the objective cost function reported in Brind et al [5] (see also section 3.1),
- the number of jobs completed (and not completed) - this takes no account of the relative importance of jobs,
- the total duration of failed jobs — note that total duration is calculated using estimated job durations for the initial SA schedule and using actual job durations for all other cases (duration is measured in minutes)

Table 1 Factors used in preliminary experimental comparisons.

Category	Description	Value
Jobs	Total jobs (see section 6.1 and Fig 8) Classes of jobs Mutually exclusive skills (% of job set with skill in brackets)	216 Appointment (100%) Commitment (0%) SK1 (13%), SK2 (62%), SK3 (25%)
Engineers	Number of engineers (variable) Skills (percentage of workforce with skill in brackets) ... Start of working day End of working day Maximum overtime allowed	30, 45 or 60 SK1 (100%), SK2 (100%), SK3 (35%) 8.00 a.m. — 8.30 a.m. 4.15 p.m. — 5.15 p.m. 1 hour per engineer
Geography	Region Postcodes	Northamptonshire NN2—NN11, NN14—NN16
General	Simulation speed Predictive scheduling algorithm Time allowed for predictive scheduling WMS RTA/predictive scheduler executed on WEST executed on	3 x real time Simulated annealing 10 CPU-minutes Sun Sparc 2 Sun Sparc LX

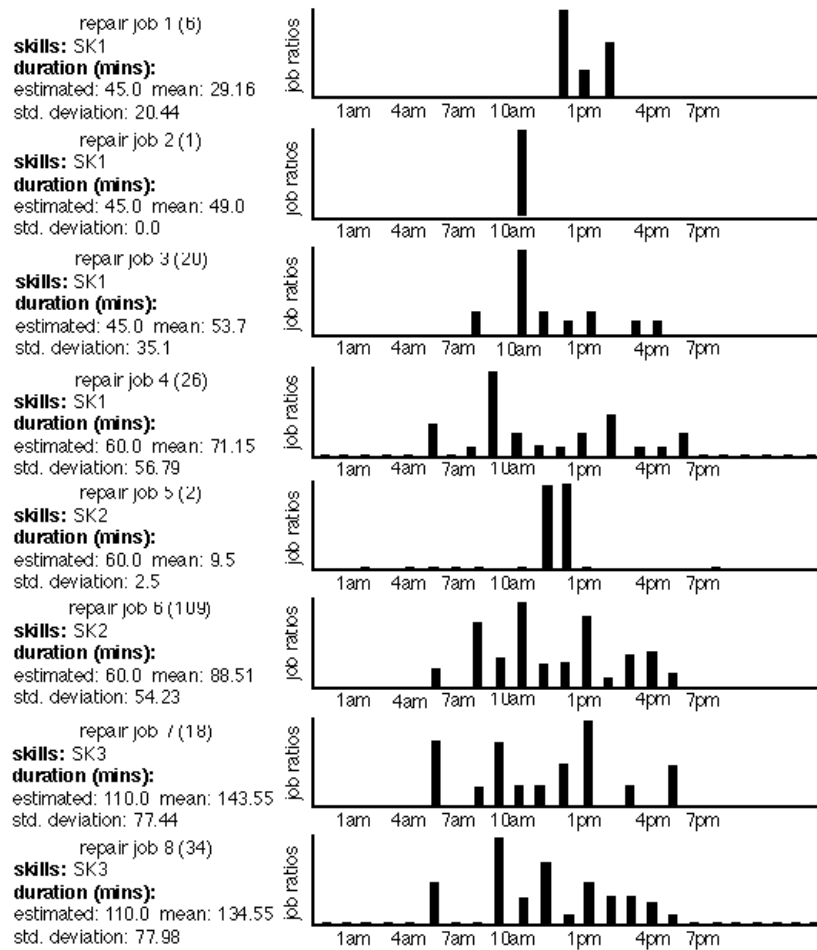


Fig 8 Job profiles.

Table 2 How good are the resulting schedules for 216 jobs?

Measure of goodness	SA predictive schedule			SA actual schedule			WMS actual schedule			SA perfect predictive schedule		
Engineers available	30	45	60	30	45	60	30	45	60	30	45	60
1. Engineers used	30	32	32	30	32	32	30	45	60	30	37	37
2. Cost (X 10 ⁶)	0.33	0.0	0.0	4.81	4.76	5.06	5.02	2.24	1.27	21.6	1.39	1.39
3. Jobs completed	211	216	216	130	130	134	141	183	207	200	207	207
Jobs not completed	5	0	0	86	86	82	75	33	9	16	9	9
4. Duration of failed jobs	550	0	0	8011	7928	8432	8359	3727	2115	3607	2308	2308
5. Travel per tech (km)	75.0	64.8	61.3	74.5	74.0	70.4	44.3	39.5	31.6	79	68	36
6. Utilisation/all (%)	95.8	66.3	49.7	94.7	72.6	52.3	93.5	86.4	70.6	100	77.2	57.9
Utilisation/not idle (%)	95.8	93.2	93.2	94.7	98.2	92.8	93.5	86.4	70.6	100	93.9	93.9
7. Grade of service (%)	97.7	100	100	60.2	60.2	62.0	65.3	84.7	95.8	92.6	95.8	95.8

- the mean travel per engineer (measured in kilometres),

- the mean travel per engineer (measured in kilometres),
- utilisation of engineers (see section 3.2) — note that some SA schedules use only part of the workforce and, therefore, utilisation is given as a percentage of both the entire workforce, and that part of the workforce which is given jobs,
- the grade of service (see section 3.2) — 100% grade of service indicates that all jobs are done.

5.2.1 Engineer activity profiles

Figure 9 shows the activity profiles for workforces of 30, 45 and 60 engineers. The left-hand profiles are achieved using the simulated annealer for scheduling, and the right-hand profiles using the WMS real time algorithm.

The shapes of the engineer activity profiles are explained as follows. The day is broken into four time periods (as tabulated in section 4.4 and shown graphically in Fig 9). Jobs cannot be allocated outside the time period in which their appointment falls. Therefore, slack periods (i.e. with many idle engineers) may occur towards the end of each time period, especially when the problem is over-resourced. This is most acute in the case where WMS has 60 engineers.

6. Examination of experimental results

The following issues have arisen as a result of these preliminary experiments.

6.1 Unused engineers in the SA experiments

A characteristic of the SA predictive scheduler is that only a partial set of engineers is scheduled if there are insufficient jobs for the whole workforce. There are 13 redundant engineers in the 45 engineer problem, and 28 redundant engineers in the 60 engineer problem. This places a lower-bound on the graphical plot of idle engineers in Fig 9.

Since the predictive SA scheduler is run once only at the start of the day, the job allocator has no means to re-introduce redundant engineers later. During the course of the day, scheduled jobs may fail because earlier jobs have overrun. This could have been avoided if it were possible to re-allocate jobs to otherwise redundant engineers. The experiments with 45 or 60 engineers are therefore unfairly biased in favour of the WMS RTA, because the RTA was able to use the entire pool of engineers. A fairer comparison is made by running the RTA with the same subset of 32 engineers chosen by the SA because then both have the same theoretical limits on factors such as the maximum duration of jobs completed. The results for this experiment

are given in Table 3 and the associated engineer activity profile is shown in Fig 10.

It can be seen from Table 3 that similar performance is obtained from both schedulers. More jobs are completed by WMS than the SA scheduler, but the Strathclyde cost function is better for the SA scheduler because longer duration jobs are completed successfully. Interestingly, WMS manages to keep travelling distances small, but the SA scheduler returns relatively large distances. This is a characteristic of the Strathclyde cost function used by the SA scheduler (see section 3.1) which is heavily biased towards reducing the duration of failed jobs. The result is that travel times are of a relatively low priority.

The difference between WMS performance when it is given the whole engineer set and when it is given a subset of engineers can be considered as a typical improvement that might be achieved when dynamic rescheduling is introduced to the SA-based job allocator. In Table 3 the performance of the WMS, based on the Strathclyde cost function, deteriorates from a score of 1.27 (see Table 2) to 4.89, demonstrating significant room for improvement.

Given that the SA-based scheduler has a tendency to allocate work to a subset of the workforce only, alternatives to rescheduling become possible in a dynamically changing environment [8]. The engineers who are initially allocated no jobs can be viewed as a resource for performing jobs which are about to go into jeopardy status. Jeopardy is reached when it is apparent that a job will fail due to earlier jobs overrunning. Jobs in jeopardy can be removed from the original engineer's schedule and reallocated to one of the unused engineers without any rescheduling being performed. This is a function of the job-allocation mechanism (see section 4).

6.2 A lower bound on job allocations

Consider the actual schedules measured by running WEST on the SA's predictive schedules. Real time job allocations are limited to the sequences generated by the SA at the start of the day and based on estimated job durations. No updates to the schedule can be made when actual job durations differ from the estimates. Thus the final measured schedules represent a worst-case performance for the SA scheduler.

6.3 An upper bound on job allocations

Consider the predictive schedules produced when the SA was given actual job durations. The SA scheduler has perfect advance knowledge of all jobs and is able to perform global optimisation over the whole day. The results presented in Table 2 therefore suggest an upper bound on the number of jobs that can be performed.

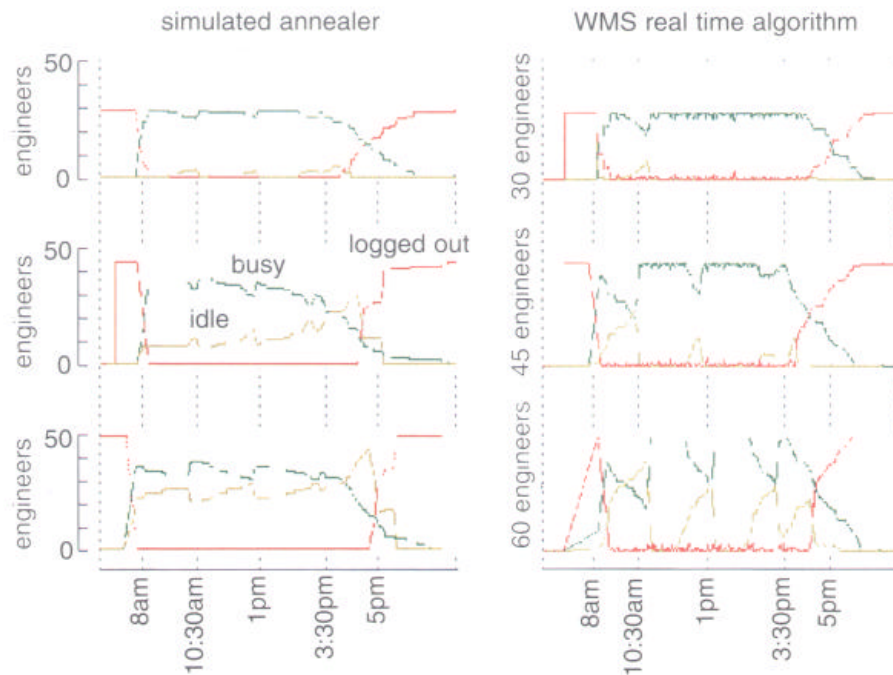


Fig 9 Experimental activity profiles for engineers attempting 216 jobs.

Table 3 Results of running the WMS with a subset of 32 engineers.

Measure of goodness	SA actual schedule 45	WMS actual schedule 32 of 45
Engineers available		
1. Engineers used		
2. Cost (X 10 ⁶)		
3. Jobs completed Jobs not completed		
4. Duration of failed jobs		
5. Travel per tech (km)		
6. Utilisation/all (%) Utilisation/not idle (%)		
7. Grade of service (%)		

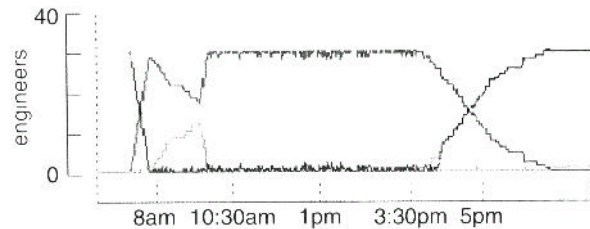


Fig 10 WMS experimental activity profile for 32 engineers attempting 216 jobs.

One caveat, however, is that an actual job duration may be larger than the time window in which the job is to be allocated. Time constraints are thus violated and the offending job can never be allocated. This has occurred for 9 jobs out of the total of 216.

6.4 Improving schedules by including dynamic capability

The upper bound on SA performance is significantly better than that achieved either with the purely predictive SA or with the WMS RTA. The only time the WMS RTA performs better than the upper bound (according to the Strathclyde cost function) is in the over- resourced, 60 engineer case. However, the SA has used only a fraction of the workforce and suffers from the time-constraint violation described above. The WMS RTA is able to schedule jobs which subsequently break time constraints (e.g. jobs may require more overtime than normally authorised) because it is working with estimates of job durations and therefore is ignorant of broken time constraints. WMS is able to reduce the effects of increased job duration by utilising the notion of maximum expected variability [1]. This is based on past experience and time constraints will only be broken when actual job durations differ by more than might be expected by analysis of historical records.

Despite the time-constraint problem, the upper bound on SA performance is better than that achieved with the existing algorithm in the critically resourced, and underresourced problem. In the 45 engineers case, the SA still only uses 37 engineers and yet achieves a cost score of 1.39 (scheduling 207 jobs) in comparison with the WMS RTA which achieves a cost score of 2.24 (scheduling 184 jobs). This indicates a possible improvement of 38% in the cost score and 13% improvement in the number of jobs allocated. The remaining cost metrics exhibit similar improvements.

7. Conclusions

Sufficient evidence has been gathered to suggest that it would be effective to employ a predictive scheduling algorithm as the basis for dynamic scheduling. The predictive scheduler has performed reasonably well on a realistic, dynamically changing problem. With the addition of dynamic rescheduling, much better results are expected.

A theoretical upper bound on the performance of such a scheduler has been calculated and promises improvements over the existing system. It will be interesting to see how close it is possible to get to the theoretical limits by adding a dynamic rescheduling capability to the SA-based allocator. This question will be investigated in future work.

References

- 1 Laithwaite R: 'Work allocation challenges and solutions in a large-scale work management environment', BT Technol J, 13, No 1, pp 46—54 (January 1995).
- 2 Thurston P and Parrott D J: 'WEST user manual', BT Laboratories, Martlesham Heath, Ipswich, IP5 7RE (1994).
- 3 Prosser P, Muller C and Brind C: 'A study of stochastic search techniques applied to vehicle routing problems', BT report, University of Strathclyde (February 1994).
- 4 Prosser P, Muller C and Brind C: 'A study of stochastic search techniques applied to vehicle routing problems: part 2', BT report, University of Strathclyde (June 1994). Brind C, Muller C and Prosser P: 'Stochastic techniques for work management', BT Technol J, 13, No 1, pp 55—63 (January 1995).
- 5 Manning M R W and Gell M A: 'A neural network assignment model', BT Technol J, 12, No 2, pp 50—656 (April 1994).
- 6 Azarmi N and Jiang Y: 'Strategies for constraint-based reactive scheduling', IRESS Report-94-2, IC-Parc, Imperial College, London (February 1994).
- 7 Azarmi N: 'A framework for reactive scheduling', Technical Report, IC-Parc, Imperial College, London (January 1995).



David Parrott received an MA degree in Engineering and Computer Science (University of Cambridge) and a PhD in Computer Science (UCL London) in 1993.

He joined BT's Intelligent Systems Unit in May 1993 and since then has been working on the application of AIP techniques to BT resource management problems.



Nader Azarmi received a BSc Honours degree in Computer Science and Mathematics from the University of Manchester in 1982, and an MSc in Computer Studies (AI) in 1985 and an MPhil in Computer Science (Logic Databases) in 1990 from the University of Essex. Since 1992 he has been pursuing a PhD degree in Computer Science (Reactive Scheduling Systems) at Imperial College, London University. In 1988 he took a senior lectureship post in the Computer Science and Electronic Engineering School of Kingston University. Since December 1989 he has been working at BT Laboratories

on the application of AIP techniques to telecommunications network automatic fault diagnosis, and to resource management.